NED University of Engineering & Technology

Department of Cyber Security · Final Year · Batch 2023

CT-486 (Network and Information Security)

Complex Computing Problem (CLO3)

VH-Cascade Cipher

Design, Implementation and Cryptanalysis of a Two-Stage Classical Product Cipher (Vigenère → Hill)

Group Member	Roll No.
Muneeb Ali Shah	CR-23012
Muneeb Ahmed	CR-23021
S.M. Faizan Hyder	CR-23026
Kabeer Ahmed	CR-23034

Table of Contents

S. No.	Description	Page No.
1	Introduction and Requirement Compliance	2
2	Cipher Design	2-4
3	Encryption and Decryption Algorithms	4-6
4	Security Analysis	6-7
5	Attack Methods	7-8
6	Attack Results	8-12
7	Efficiency and Performance	12-13
8	Suggestions for Improving Security	14
9	Limitations of the Study	14-15
10	Conclusion	15
11	References	15

1. Introduction and Requirement Compliance

Classical ciphers such as Caesar (Shift), Vigenère and Hill are no longer used to protect real data, but they remain the best way to learn the vocabulary of cryptanalysis: key space, frequency statistics, the index of coincidence, known-plaintext leakage and the difference between *looking* random and *being* secure. This report documents a custom cipher, the **VH-Cascade**, built from two classical techniques, together with the attacks we wrote to break it and the measurements that describe both.

The cipher first applies a **Vigenère** polyalphabetic substitution (Stage 1) and then a **2×2 Hill** matrix cipher (Stage 2). Both operate on the 26-letter alphabet A–Z, so every letter is a number in Z_{26}. The work follows the brief of the Complex Computing Problem: design, implement, attack, analyse efficiency, and suggest improvements. Table 1 maps each requirement of the brief to the section of this report (and the source file) that satisfies it.

Table 1. Requirement compliance matrix

Requirement in the brief	How it is met	Where
Combine two or more classical ciphers	Vigenère (polyalphabetic) followed by Hill (polygraphic, matrix-based)	§2, `cipher.py`
At least two stages, each a different technique	Stage 1 = additive keyword substitution; Stage 2 = linear digraph transform	§2.2–2.3
Key length of at least 10 characters	Vigenère keyword ≥ 10 letters (enforced in code) + 4-letter Hill key	§2.5, `validate_key()`
Encrypt and decrypt, inputs of any length, A–Z	Input cleaned to A–Z; odd lengths padded with X; tested up to 1,000,000 letters	§3, §8
Break it by frequency analysis and/or known-plaintext	Both implemented: ciphertext-only IC/frequency attack and known-plaintext linear-algebra attack	§6, `attacks.py`
Success rate and effort for various lengths	Monte-Carlo experiments (30–100 trials per point), time and candidates counted	§7
Time complexity; comparison with Shift cipher	$\Theta(n)$ analysis plus measured timings against Shift up to 10^6 letters	§8
Security analysis and improvement suggestions	Key space, statistical flattening, structural weaknesses; five low-cost improvements	§5, §9
Code on GitHub	Repository with all scripts, data and figures	Cover page

2. Cipher Design

2.1 Overview

A *product cipher* applies two or more simple ciphers in sequence. Shannon (1949) observed that a good product combines ciphers that supply different properties: **confusion** (the relationship between key and ciphertext is complicated) and **diffusion** (the influence of one plaintext letter is spread over many ciphertext letters). VH-Cascade follows that idea in its classical form: Vigenère hides the letter frequencies of the language, and Hill spreads every letter over a pair of ciphertext letters. Figure 1 shows the data flow.

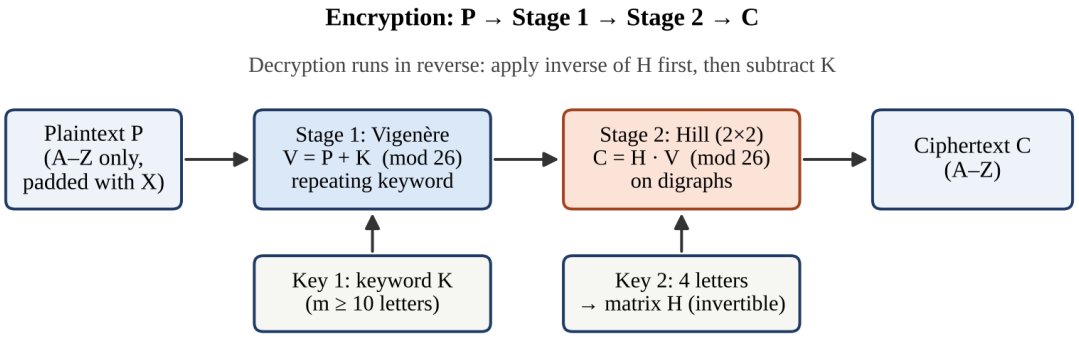


Figure 1. Data flow of the VH-Cascade cipher (encryption left to right).

2.2 Stage 1 – Vigenère substitution

The plaintext is converted to numbers (A = 0, …, Z = 25). A keyword K of m ≥ 10 letters is repeated along the message and added letter by letter:

$$V_i = (P_i + K_{i \bmod m}) \bmod 26$$

Because the same plaintext letter is shifted by different amounts depending on its position, a given letter such as E is mapped to up to m different ciphertext letters. This flattens the single-letter frequency distribution, which is the weakness that kills the plain Shift cipher.

2.3 Stage 2 – Hill matrix cipher

The output V of Stage 1 is cut into digraphs (pairs of letters) and each pair is multiplied by a secret 2×2 matrix H modulo 26:

$$[C_{2t}, C_{2t+1}]^T = H \cdot [V_{2t}, V_{2t+1}]^T \bmod 26, \quad H = [[a, b], [c, d]]$$

H is derived from a 4-letter key read row by row (for example HILL gives H = [[7, 8], [11, 11]]). For decryption to exist, H must be invertible modulo 26, which holds exactly when gcd(det H, 26) = 1, that is, when det H is odd and not a multiple of 13. The code rejects any Hill key that violates this. Among the 26^4 = 456,976 possible 4-letter keys, exactly **157,248** give an invertible matrix.

2.4 Why this combination

- **Complementary roles.** Vigenère works on single positions and hides the language statistics; Hill works on neighbouring pairs and removes the letter-by-letter alignment that Kasiski or Friedman tests on a Vigenère ciphertext rely on.
- **Both are taught in class** and use the same modular arithmetic over Z_{26}, so one set of helper functions (modular inverse, matrix inverse) serves both stages.
- **Cheap to run.** Each stage is a single pass over the message, so the cost stays linear (Section 8).
- **Distinct techniques.** One is a polyalphabetic *substitution*, the other a polygraphic *linear transformation*; this meets the “different classical technique per stage” rule.

2.5 Key structure

The secret key is the pair (K, H-key): a Vigenère keyword of at least 10 letters and a 4-letter Hill key. Table 2 compares the nominal key space with the single-stage ciphers.

Table 2. Nominal key space

Cipher	Key space	≈ bits
Shift	26	4.7

Cipher	Key space	≈ bits
Vigenère only, m = 10	$26^{10} \approx 1.4 \times 10^{14}$	47.0
Hill 2×2 only	157,248	17.3
VH-Cascade, m = 10	$26^{10} \times 157{,}248 \approx 2.2 \times 10^{19}$	64.3
VH-Cascade, m = 13 (default key CYBERSECURITY)	$\approx 3.9 \times 10^{23}$	78.4
VH-Cascade, m = 16	$\approx 6.9 \times 10^{27}$	92.5
VH-Cascade, m = 20	$\approx 3.1 \times 10^{33}$	111.3

The nominal figures look comfortable, but Section 5 shows that they say little about the real difficulty of breaking the cipher, because the two keys can be attacked separately.

3. Encryption and Decryption Algorithms

3.1 Pre-processing

Input text is upper-cased and every character outside A–Z (spaces, digits, punctuation) is dropped. If the remaining length is odd, the padding letter X is appended so that the Hill stage always receives complete digraphs. The Vigenère keyword is cleaned in the same way and rejected if shorter than 10 letters.

3.2 Encryption

```
ALGORITHM ENCRYPT(plaintext, vig_key, hill_key)
 1. K <- numbers(clean(vig_key));  if |K| < 10 then error
 2. H <- 2x2 matrix from hill_key; if gcd(det(H),26) != 1 then error
 3. P <- numbers(clean(plaintext));  if |P| is odd then append X (=23)
 4. m <- |K|
 5. for i = 0 .. |P|-1:                          // Stage 1: Vigenere
        V[i] <- (P[i] + K[i mod m]) mod 26
 6. for t = 0 .. |V|/2 - 1:                      // Stage 2: Hill
        C[2t]   <- (H00*V[2t] + H01*V[2t+1]) mod 26
        C[2t+1] <- (H10*V[2t] + H11*V[2t+1]) mod 26
 7. return letters(C)
```

3.3 Decryption

Decryption undoes the stages in the reverse order. The inverse of a 2×2 matrix modulo 26 is the adjugate multiplied by the modular inverse of the determinant:

$$H^{-1} = (\det H)^{-1} \cdot [[d, -b], [-c, a]] \mod 26$$

```
ALGORITHM DECRYPT(ciphertext, vig_key, hill_key)
 1. K, H <- as in ENCRYPT;   require |ciphertext| even
 2. D <- det(H) mod 26;  Dinv <- D^-1 mod 26
 3. Hinv <- Dinv * [[H11, -H01], [-H10, H00]]  mod 26
 4. C <- numbers(clean(ciphertext))
 5. for t = 0 .. |C|/2 - 1:                      // undo Stage 2
        V[2t]   <- (Hinv00*C[2t] + Hinv01*C[2t+1]) mod 26
        V[2t+1] <- (Hinv10*C[2t] + Hinv11*C[2t+1]) mod 26
 6. for i = 0 .. |V|-1:                          // undo Stage 1
        P[i] <- (V[i] - K[i mod m]) mod 26
 7. return letters(P)     // a trailing X may be padding
```

3.4 Worked example

The program's demonstration run encrypts "Meet me at the NED gate at noon" with the default keys CYBERSECURITY and HILL. After cleaning, the plaintext is

MEETMEATTHENEDGATEATNOON (24 letters), and the key stream is the 13-letter keyword repeated:

```
Plaintext   P : MEETMEATTHENEDGATEATNOON
Key stream  K : CYBERSECURITYCYBERSECURI
Stage 1     V : OCFXDWEVNYMGCFEBXVSXPIFV      (V = P + K mod 26)
Stage 2     C : KULWPPOPXRCQCZKDRQYJNTVA      (C = H.V mod 26)
H = [[7, 8], [11, 11]]      H^-1 = [[25, 22], [1, 23]]
```

For the first digraph, ME = (12, 4) and the key digraph CY = (2, 24), so V = (14, 28 mod 26) = (14, 2) = “OC”. Stage 2 gives $7\cdot14 + 8\cdot2 = 114 \equiv 10$ (K) and $11\cdot14 + 11\cdot2 = 176 \equiv 20$ (U), so the ciphertext digraph is “KU”. Decryption reverses it: $H^{-1}\cdot(10, 20) = (690, 470) \equiv (14, 2)$ = “OC”, and subtracting the key (2, 24) returns (12, 4) = “ME”. Here $\det H = 7\cdot11 - 8\cdot11 = -11 \equiv 15 \pmod{26}$, $\gcd(15, 26) = 1$, and $15^{-1} \equiv 7$, which produces the H^{-1} shown above.

4. Implementation

The solution is written in Python 3 with NumPy (vectorised attack code) and Matplotlib (figures). The repository is organised as follows.

Table 3. Project files

File	Purpose
`cipher.py`	Text helpers, modular inverse, 2×2 matrix inverse, Vigenère, Hill, `encrypt`, `decrypt`, `encrypt_trace`, and the Shift cipher baseline; command-line interface
`attacks.py`	`known_plaintext_attack`, `frequency_analysis_attack`, `shift_brute_force`, and scoring helpers (index of coincidence, χ^2, best shift, bigram score)
`main.py`	End-to-end demo: encrypt, decrypt, then break the cipher both ways
`experiments.py`	Monte-Carlo experiments for success rate, effort and timing; writes `results/results.json`
`make_charts.py`	Produces the report figures from `results.json`
`data/sample_plaintext.txt`	Public-domain English text (G. K. Chesterton, *The Man Who Was Thursday*, Project Gutenberg), about 249,000 letters after cleaning

Usage examples: `python main.py` runs the demo; `python cipher.py enc "attack at dawn" --vkey CYBERSECURITY --hkey HILL` encrypts a message; `python experiments.py && python make_charts.py` reproduces every number and figure in this report. The code validates keys, rejects non-invertible Hill matrices and keywords shorter than 10 letters, and handles any message length (tested from 20 letters to 1,000,000 letters).

4.1 Demonstration output

Running `main.py` with the default keys and a 1,000-letter excerpt gave the following results (the first block was shown in Section 3.4).

```
KNOWN-PLAINTEXT ATTACK (40 known letters)
  Recovered H = [[7, 8], [11, 11]],  Vigenere key = CYBERSECURITY (length 13)
  Full message recovered: True   | time 0.78 ms

CIPHERTEXT-ONLY FREQUENCY ANALYSIS ATTACK (1,000 letters)
  Recovered H = [[7, 8], [11, 11]],  Vigenere key = CYBERSECURITY
  Letters correct: 100.0%  | candidates scored 10098  | time 567 ms
  First 80 letters: NAKYTWISTSWENTSTRAIGHTUPASIFFROMAFACTORY...
```

Both attacks recovered the complete key without being told its length. The recovered text is the opening of Chesterton's novel, as expected.

5. Security Analysis

5.1 Resistance to naive statistical attacks

A direct letter-frequency attack on a Shift or simple substitution cipher works because the ciphertext keeps the peaks of English (E, T, A, O …). Figure 2 compares the expected English distribution with the VH-Cascade ciphertext of a 3,000-letter passage. The ciphertext histogram is almost flat: the most frequent ciphertext letter has a share of 5.07 % and the rarest 3.03 %, against 12.2 % and 0 % in the plaintext sample.

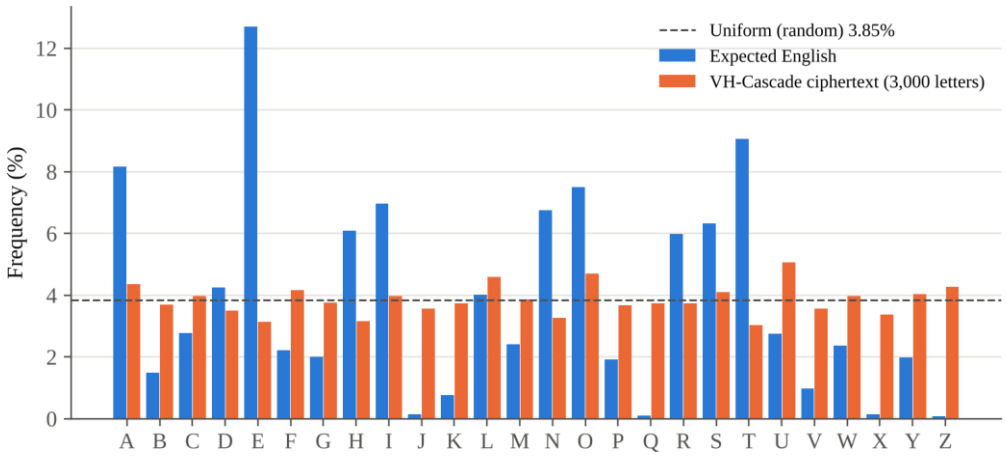


Figure 2. Letter frequencies: expected English versus VH-Cascade ciphertext (3,000 letters).

The index of coincidence (IC) quantifies the same effect. For the 3,000-letter passage we measured IC = 0.0636 for the plaintext (theory for English: 0.0655), 0.0417 after Stage 1 alone and **0.0387** for the final ciphertext, which is practically the value of uniformly random text (1/26 = 0.0385). Hence the two standard first moves, a one-alphabet frequency count and a Friedman/Kasiski period test on the raw ciphertext, give no information. The Hill stage is what removes the last trace of period structure in the raw stream, because each ciphertext letter now depends on two Vigenère outputs with different key letters.

5.2 Structural weaknesses

Statistical flatness is necessary but not sufficient. The cipher has three structural weaknesses that our attacks exploit.

1. **Everything is linear.** Writing the key stream as digraphs K_t, encryption is $C_t = H\cdot(P_t + K_t) = H\cdot P_t + H\cdot K_t$. Both stages are affine maps over Z_{26}, so the cascade is itself just a Hill cipher with a periodic additive term. Shannon's lesson applies: composing two ciphers from the same algebraic family does not create a stronger cipher of a new family. Linear algebra solves it.
2. **The key stream is periodic.** The Vigenère key repeats every m letters, so the digraph key repeats every q = m / gcd(m, 2) digraphs. For two digraphs q apart, $K_{t+q} = K_t$, so the key cancels in $C_{t+q} - C_t = H\cdot(P_{t+q} - P_t)$. That removes the Vigenère key completely and leaves only the 4 unknown entries of H.
3. **The two keys can be attacked separately.** Each row of H^{-1} applied to the ciphertext digraphs produces a stream that equals English plus a periodic shift, whose column IC is about 0.066; a

wrong row gives about 0.038. Thus the 504 admissible rows (of 676 pairs, those with gcd(r_0, r_1, 26) = 1) can be tested one by one instead of testing 157,248 matrices together.

5.3 Nominal versus effective security

The nominal key space of the default key is about 2^{78}. The ciphertext-only attack scores only 10,098 candidates (rows × periods plus 18 final candidates), roughly $2^{13.3}$, and the known-plaintext attack tests at most 30 key lengths. The effective work is therefore tens of orders of magnitude below the nominal key space. Table 4 summarises the attack surface.

Table 4. Attack surface summary

Scenario	What the attacker needs	Observed outcome
Ciphertext only	≈ 800 letters of English ciphertext	95 % full recovery at 800 letters, 100 % at 3,000; about 0.4–1.1 s
Known plaintext	≈ 40 known letters at the start of the message	99 % full recovery with 40 letters; under 1 ms
Shift cipher (baseline)	≈ 50 letters of ciphertext	≥ 97.5 % success from 50 letters; about 0.2 ms

In other words, the cascade needs roughly 16 times more ciphertext than the Shift cipher before a ciphertext-only attack reliably succeeds, but it still falls to a short, fast, fully automatic attack. The design goal of the brief (build a cipher and then break it) is met, and the analysis in Section 9 shows how to move the weak points at low cost.

6. Attack Methods

6.1 Known-plaintext attack (KPA)

Assumption. The attacker holds the full ciphertext and knows the first L letters of the plaintext (for example a standard message header). The attacker does not know the key length m, the keyword or the matrix H.

Idea. From Section 5.2, for two digraphs q = m / gcd(m, 2) apart the key cancels:

$$C_{t+q} - C_t = H \cdot (P_{t+q} - P_t) \pmod{26}$$

Every such pair of known digraphs yields two linear equations for the four entries of H. Because 26 = 2 × 13 is not prime, Z_{26} is not a field, so the system is solved modulo 2 and modulo 13 separately (each time using two linearly independent difference vectors) and the two answers are merged with the Chinese Remainder Theorem.

Algorithm: known-plaintext attack

```
for m = 1 .. 30:                                    // unknown key length
    q <- m / gcd(m, 2)
    need at least q + 4 known digraphs, else skip
    dP[t] <- P[t+q] - P[t];   dC[t] <- C[t+q] - C[t]     (mod 26)
    H2  <- solve  H.dP = dC  mod 2;   H13 <- solve it mod 13
    H   <- CRT(H2, H13);  reject if gcd(det H, 26) != 1
    reject if H.dP != dC for any t                  // verification
    Hinv <- H^-1;  K[(2t+j) mod m] <- (Hinv.C[t])[j] - P[t][j]
    reject if two known digraphs disagree on a key letter
    return H, K and decrypt the whole ciphertext
```

The first m that passes every consistency check is the key length. The number of candidates tried therefore equals the true m, which explains the mean of about 13.3 candidates in Table 5 (the experiment draws m uniformly from 10 to 16).

Minimum data. The implementation demands at least q + 4 known digraphs (two to fix H, two extra as checks), that is **2(q + 4)** known letters. This is 18, 20, 22, 24 letters for m = 10, 12, 14, 16, but 30, 34, 38 for the odd lengths 11, 13, 15, because an odd m has q = m while an even m has q = m / 2. The experiments in Section 7 confirm this prediction.

6.2 Ciphertext-only attack by index of coincidence and frequency analysis

Assumption. The attacker holds only ciphertext (English text) of several hundred letters or more and knows nothing about the keys, except an upper bound of 20 on the keyword length used in our implementation.

Idea. Divide and conquer: break Stage 2 first, row by row, using the IC; then break the remaining Vigenère with the classical method.

1. **Enumerate rows.** Build all 504 pairs (r_0, r_1) with $\gcd(r_0, r_1, 26) = 1$, and compute the stream $S_t = r_0 \cdot C_{2t} + r_1 \cdot C_{2t+1} \bmod 26$ for each (a NumPy matrix of 504 × n/2).
2. **IC test per row and period.** For each candidate period q = 1…20, split the stream into q columns and average their ICs. A correct row of H^{-1} gives a stream of English with a periodic shift, so its column IC at the true period is about 0.066; a wrong row stays near 0.038. This is 504 × 20 = 10,080 evaluations.
3. **Choose two independent rows.** Keep the best row, then the best-scoring row that is not a unit multiple of it and whose determinant with the first is invertible mod 26. (Rows that differ by a unit multiplier share the same IC.)
4. **Fix the scale.** For each chosen row, try the 12 invertible multipliers modulo 26, remove the best Caesar shift of every column by correlating with English letter frequencies, and rank by χ^2 distance to English. Keep the top 3 scalings per row.
5. **Assemble and verify.** Try every combination of scalings and both row orders (3 × 3 × 2 = 18 matrices). For each, apply H^{-1}, estimate the Vigenère period from the IC, recover each key letter by frequency correlation, decrypt, and keep the candidate with the highest share of the 30 most common English bigrams.

The total effort is a fixed 10,080 + 18 = **10,098** candidate evaluations, whatever the message length, which matches the `mean_evaluations` column of the experiment.

6.3 Baseline: Shift cipher brute force

For comparison the Shift cipher is broken by trying all 26 keys and choosing the shift whose decryption has the smallest χ^2 distance to English letter frequencies.

7. Attack Results

All experiments use random keys (Vigenère keyword of random letters, random invertible Hill key) and random excerpts of the Chesterton text; the random seed is fixed at 486 for reproducibility. A KPA trial succeeds only if the full plaintext is recovered exactly. A ciphertext-only trial counts as a success when at least 99 % of the letters are correct; the mean fraction of letters recovered is reported as well. The test machine was an x86-64 system with 2 CPUs running Python 3.13.

7.1 Known-plaintext attack

Table 5 and Figure 3 give the success rate against the number of known letters (100 trials per row, 1,000-letter messages, keyword length drawn uniformly between 10 and 16).

Table 5. Known-plaintext attack versus amount of known plaintext

Known letters	Full recovery	Mean time (ms)	Mean candidates (m) tried
10	0 %	0.12	30.0
16	0 %	0.17	30.0
20	28 %	0.26	24.7
24	57 %	0.36	20.4
30	61 %	0.41	19.3
36	86 %	0.67	15.0
40	99 %	0.74	13.3
50	100 %	0.82	13.3
60	100 %	0.60	13.3

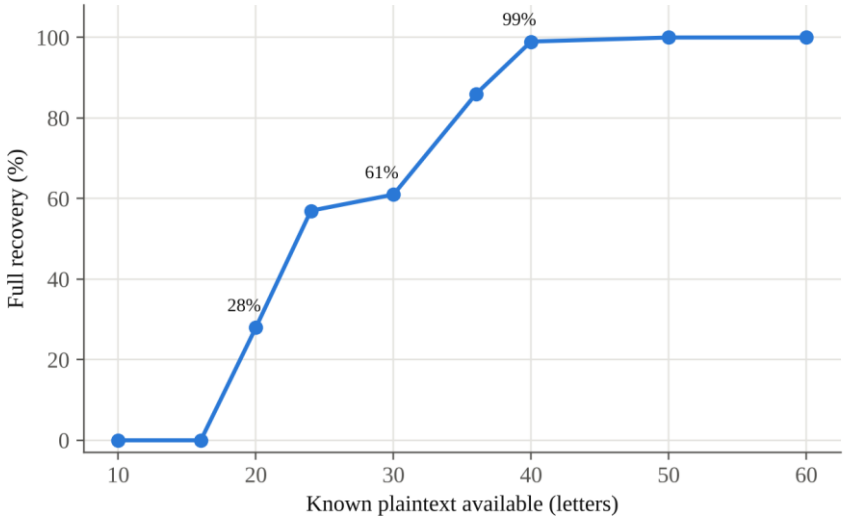


Figure 3. Known-plaintext attack: probability of full recovery versus known plaintext length.

Nothing is recovered with 10 or 16 known letters, since even the shortest key needs 18. Success then rises as a staircase: at 20 letters only the keys of length 10 and 12 are within reach (28 % observed); at 24 letters all even lengths 10–16 are covered (57 %, close to the 4 out of 7 equally likely lengths); at 36 letters most odd lengths join (86 %); and at 40 letters every key is broken (99 %), with 100 % from 50 letters on. The attack takes under one millisecond in all cases.

7.2 Effect of keyword length on the known-plaintext attack

Table 6 and Figure 4 measure the smallest even-length prefix that recovers the whole message for each fixed keyword length (60 trials each). The measured means agree with the predicted minimum 2(q + 4).

Table 6. Known plaintext needed per keyword length

Key length m	q = m / gcd(m,2)	Predicted minimum 2(q+4)	Measured mean	Measured max
10	5	18	18.3	24
11	11	30	30.6	38
12	6	20	20.9	30
13	13	34	35.1	50

Key length m	q = m / gcd(m,2)	Predicted minimum 2(q+4)	Measured mean	Measured max
14	7	22	22.5	28
15	15	38	38.6	48
16	8	24	24.4	30
20	10	28	28.9	36

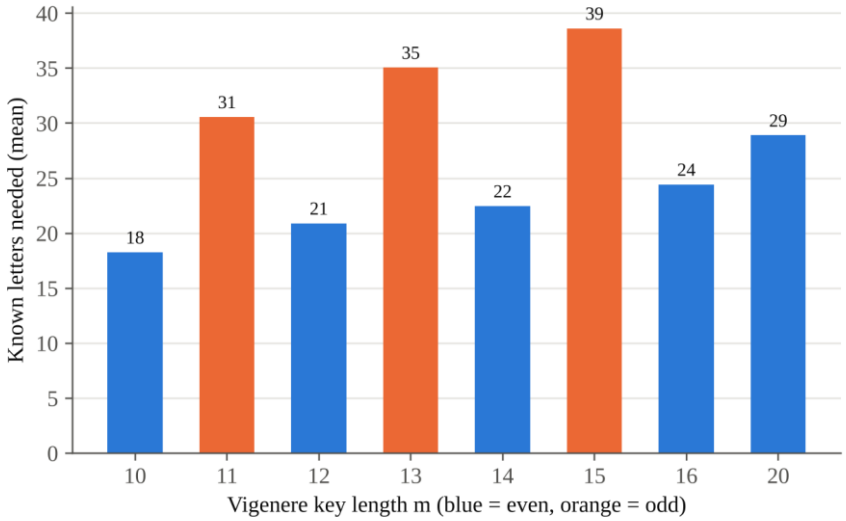


Figure 4. Mean known plaintext needed by keyword length (blue = even m, orange = odd m).

The zig-zag in Figure 4 is the signature of q: odd keyword lengths force a period of 2m letters instead of m, and so need markedly more known plaintext (31, 35 and 39 letters against 18–24 for even lengths up to 16). A longer key helps only mildly (m = 20 needs 29 letters). With a 50-letter prefix the attack recovered the entire message at every ciphertext length tested from 100 to 3,000 letters (50 trials each); its time grows only from 0.26 ms to 1.38 ms, because the long ciphertext is decrypted once at the end.

Table 7. Known-plaintext attack with a fixed 50-letter prefix and growing ciphertext

Ciphertext length	Full recovery	Mean time (ms)
100	100 %	0.26
200	100 %	0.31
400	100 %	0.37
800	100 %	0.52
1,000	100 %	0.55
2,000	100 %	0.89
3,000	100 %	1.38

7.3 Ciphertext-only attack

Table 8 and Figure 5 show the ciphertext-only attack against ciphertext length (40 trials per row, keyword length 10–16).

Table 8. Ciphertext-only (IC + frequency) attack versus ciphertext length

Ciphertext length	Success (≥ 99 % letters)	Mean letters recovered	Mean time (ms)	Candidates scored
100	0 %	7.3 %	189	10,098

Ciphertext length	Success (≥ 99 % letters)	Mean letters recovered	Mean time (ms)	Candidates scored
200	0 %	10.2 %	208	10,098
300	12.5 %	35.8 %	225	10,098
400	45 %	69.5 %	245	10,098
600	65 %	85.7 %	309	10,098
800	95 %	98.6 %	376	10,098
1,000	95 %	98.7 %	446	10,098
1,500	95 %	98.1 %	591	10,098
2,000	97.5 %	99.4 %	780	10,098
3,000	100 %	100.0 %	1093	10,098

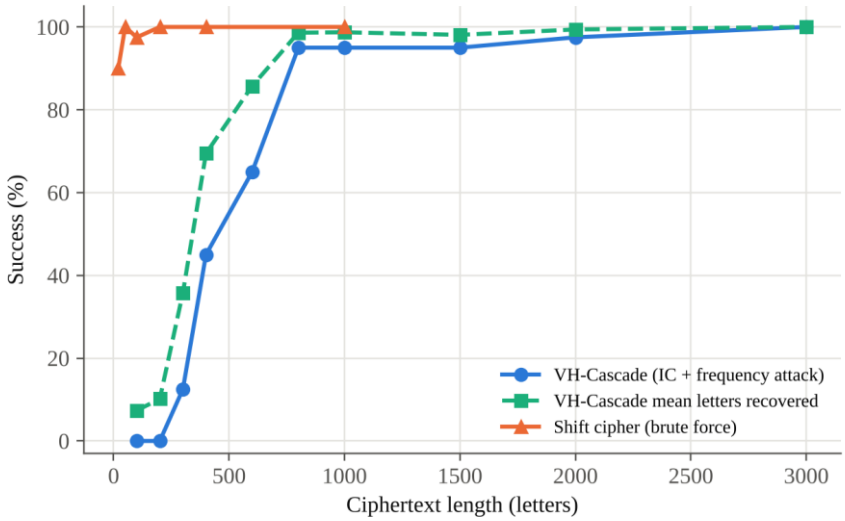


Figure 5. Ciphertext-only attack success versus ciphertext length, compared with the Shift cipher.

The attack needs statistics, so very short ciphertexts defeat it: at 100–200 letters it recovers only 7–10 % of the letters and never succeeds. The transition happens between 300 and 800 letters: 12.5 % full success at 300, 45 % at 400, 65 % at 600 and 95 % at 800, with 98.6 % of letters correct on average. Beyond 2,000 letters it is almost always perfect and at 3,000 letters it succeeded in every trial. A failed run loses a sizeable share of the text (roughly a quarter of the letters on average at 800 letters), which is why the two failures in 40 trials lower the mean accuracy to 98.6 %. The Shift cipher, by contrast, is broken from 50 letters upward (Table 10), so the cascade needs about 16 times more ciphertext before a ciphertext-only attacker is reliably successful.

The effort is constant in the number of candidates (10,098, independent of n) and grows linearly in time with the message length, from 189 ms at 100 letters to 1.09 s at 3,000 letters, since each of the 10,080 row-period tests scans the whole ciphertext.

7.4 Effect of keyword length on the ciphertext-only attack

Table 9. Ciphertext-only success versus keyword length (800 letters, 30 trials)

Key length m	Success rate
10	90 %
13	90 %
16	86.7 %

Key length m	Success rate
20	76.7 %

Longer keywords reduce the success rate (90 % at m = 10 and 13, 87 % at m = 16 and 77 % at m = 20) because each key letter is estimated from fewer samples (about n / m letters per column); with 3,000 letters the effect disappears. The trend is real but small, so lengthening the key alone is not an effective defence. With 30 trials per row, one trial equals 3.3 percentage points, so small differences between rows should not be over-interpreted.

7.5 Baseline: Shift cipher

Table 10. Shift cipher brute force (40 trials per row)

Ciphertext length	Success rate	Mean time (ms)	Keys tried
20	90 %	0.15	26
50	100 %	0.17	26
100	97.5 %	0.19	26
200	100 %	0.24	26
400	100 %	0.28	26
1000	100 %	0.51	26

The Shift cipher is broken with only 26 candidates and about 0.15–0.5 ms. Even 20 letters give 90 % success. Against that baseline, the cascade raises the effort by a factor of about 400 in candidates (10,098 versus 26), by a factor of almost 900 in time for a ciphertext of 1,000 letters (446 ms versus 0.5 ms), and by a factor of roughly 16 in the ciphertext needed, but it does not provide security in an absolute sense.

8. Efficiency and Performance

8.1 Time complexity of encryption and decryption

Let n be the message length in letters.

- **Pre-processing** (cleaning and conversion to numbers) is one pass: Θ(n). Key validation costs Θ(m) and the matrix inverse (including the modular inverse of the determinant, computed with the extended Euclidean algorithm) costs Θ(1) for the fixed modulus 26, both independent of n.
- **Stage 1 (Vigenère)** performs one addition and one reduction modulo 26 per letter, with the key index computed as i mod m: Θ(n) time.
- **Stage 2 (Hill)** processes n/2 digraphs, each with 4 multiplications, 2 additions and 2 reductions: Θ(n) time (about 2n multiplications in total).
- **Decryption** has the same two passes in reverse order plus a Θ(1) matrix inversion: Θ(n) time.
- **Space** is Θ(n) for the arrays of numbers holding the message.

Encryption and decryption are therefore both **Θ(n)**, the same order as the Shift cipher, which performs a single addition per letter. The second stage only increases the constant factor.

8.2 Measured performance against the Shift cipher

Table 11 reports the best of 3–5 repetitions for each message size, using the default keys; Figure 6 plots the same data on logarithmic axes.

Table 11. Encryption and decryption time (ms)

Letters	VH enc.	VH dec.	Shift enc.	Shift dec.	VH/Shift enc.	VH/Shift dec.
1,000	0.23	0.26	0.15	0.15	1.52×	1.71×
10,000	2.20	2.51	1.54	1.47	1.43×	1.71×
100,000	23.13	25.65	14.69	14.84	1.58×	1.73×
1,000,000	246.12	280.08	154.64	161.98	1.59×	1.73×

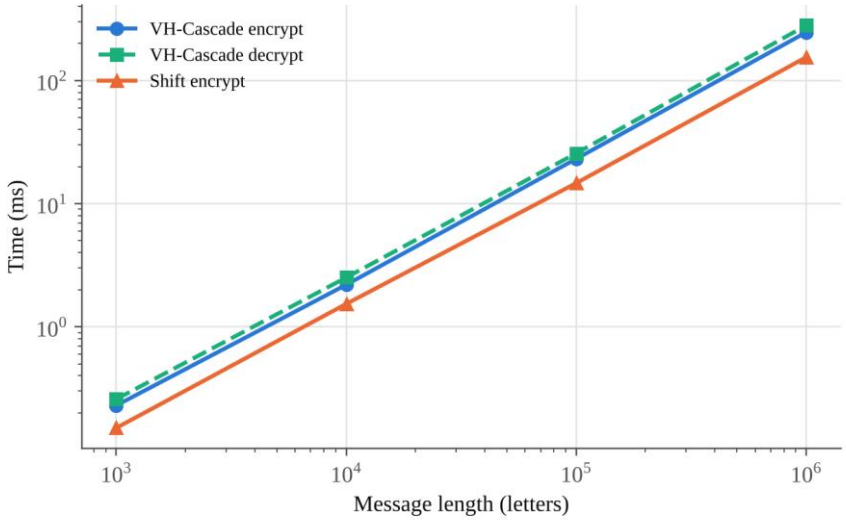


Figure 6. Encryption and decryption time versus message length (log–log scale).

All curves are parallel straight lines of slope 1 on the log–log plot, which confirms linear growth: a hundred-fold increase in message length gives a hundred-fold increase in time. The VH-Cascade is about 1.4–1.6 times slower than the Shift cipher for encryption and about 1.7 times slower for decryption, which is the expected price of an extra pass with 4 multiplications per digraph. In absolute terms, 1,000,000 letters are encrypted in about 246 ms (roughly 4 million letters per second) and decrypted in about 280 ms in pure Python, so the added cost is negligible for any practical text size. Decryption is slightly slower than encryption because it also computes the inverse matrix, and both Python implementations would be much faster in C or C++.

8.3 Computational effort of the attacks

Table 12. Effort of the attacks

Attack	Candidates	Complexity	Measured time
Shift brute force	26	$\Theta(n)$	0.15–0.5 ms
VH known-plaintext (L = 40–50)	$m \leq 30$ key lengths (mean 13.3)	$\Theta(M \cdot L + n)$, M = 30	0.6–0.8 ms; 1.4 ms at n = 3,000
VH ciphertext-only (IC + frequency)	10,098 (fixed)	$\Theta(R \cdot Q \cdot n)$, R = 504, Q = 20	189 ms (n = 100) to 1,093 ms (n = 3,000)
Brute force over all VH keys (for reference)	$\approx 2.2 \times 10^{19}$ (m = 10) to 3.9×10^{23} (m = 13)	$\Theta(2^{64} \ldots 2^{78} \cdot n)$	Infeasible

The comparison makes the central point of the analysis: the cascade is only about 1.5 times more expensive to run than the Shift cipher, and it multiplies the attack effort by a large factor (about 400 in candidates), but the attacks scale with ciphertext length like the encryption itself and remain interactive for any realistic message.

9. Suggestions for Improving Security

The suggestions below keep the cipher classical and linear in n. Each one targets a specific weakness identified in Section 5.2; none is claimed to make the cipher secure against modern cryptanalysis.

Table 13. Low-cost improvements

Improvement	Weakness it addresses	Added cost
1. Non-periodic keystream. Replace the repeating keyword by an autokey (keyword followed by the plaintext) or a running key from a keyed generator.	Removes the period q, so the key no longer cancels in $C_{t+q} - C_t$ and the IC period test loses its signal.	One array read per letter; still $\Theta(n)$.
2. Keyed substitution after Hill. Add a third stage that maps each letter through a keyword-mixed alphabet (a permutation of 26 letters).	Breaks the affine structure on which the linear-algebra attack and the row-by-row IC test both depend.	One table lookup per letter; a 26-byte table.
3. Keyed transposition between the stages. Permute letter positions (for example columnar transposition) before Hill.	Hill no longer sees fixed neighbouring pairs, so rows of H^{-1} cannot be tested in isolation.	One index permutation; $\Theta(n)$.
4. Better parameters. Use an odd keyword length m (period 2m instead of m), keep it long, and consider a 3×3 Hill matrix (about 2^{40} invertible keys instead of $2^{17.3}$).	Odd m needed 31–39 known letters against 18–24 for even m; a longer key slightly reduces ciphertext-only success; a larger matrix enlarges the key space and diffusion.	About 1.5× more multiplications per letter for 3×3; no cost for odd m.
5. Fresh keys and random padding. Never reuse a key for two messages; add random nulls instead of the fixed X, and avoid predictable headers.	Limits the known-plaintext attack (a fixed header is the ideal known plaintext) and key-reuse attacks.	Operational only.

Improvements 1–3 together would remove both attack routes used in this project. The row-by-row IC test relies on the correct row of H^{-1} turning the ciphertext into shifted English, which fails when a keyed substitution or transposition sits between that row and the plaintext. They would, however, add new targets (for instance, recovering the substitution table from long known plaintext), so the stages should be re-evaluated with new attacks after each change. Finally, such measures raise only the work of an attacker armed with letter statistics. For real data the correct engineering answer is a modern, standardised cipher such as AES in an authenticated mode, and this project should be seen as a learning exercise.

10. Limitations of the Study

- **One text source.** All plaintext is taken from a single English novel. Other texts (technical, short messages, other languages) would change the statistics, and the attacks assume the plaintext is natural English.
- **Sample sizes.** The experiments use 30–100 trials per data point, so rates have a granularity of 1–3.3 percentage points (for example, the plateau at 95 % between 800 and 1,500 letters in Table 8 corresponds to two failed trials out of 40).
- **Attack bounds.** The ciphertext-only attack searches keyword lengths up to 20 and the known-plaintext attack up to 30. An attacker can raise these bounds at a cost that grows only linearly, so keywords longer than the bound are not a defence.
- **Definition of success.** The ciphertext-only success criterion (≥ 99 % letters correct) tolerates up to 1 % wrong letters; the mean-accuracy column in Table 8 gives the complete picture.

- **Timing.** Timings are from one 2-CPU x86-64 machine with an interpreted language; absolute values will differ elsewhere, but the ratios and growth rates should hold. A repeat run of the experiments produced slightly different timings and success rates, as expected for randomised experiments; the conclusions did not change.
- **Fixed message structure.** The known-plaintext attack assumes the known letters are a prefix; the same algebra works for any known block whose position is known.

11. Conclusion

We designed VH-Cascade, a two-stage classical cipher that cascades a Vigenère substitution (keyword of at least 10 letters) with a 2×2 Hill transformation, implemented encryption and decryption for arbitrary A–Z text, and broke the cipher with two attacks that exploit its algebraic structure. The cascade does hide the letter statistics of English: the ciphertext index of coincidence is 0.0387, essentially that of random text, and simple frequency analysis fails. But because both stages are affine over Z_{26} and the key is periodic, the cipher fell to a known-plaintext attack with about 40 known letters in under a millisecond, and to a ciphertext-only attack with about 800 letters in under half a second. Encryption and decryption are $\Theta(n)$ and only 1.4–1.7 times slower than the Shift cipher.

The main lesson is that a larger nominal key space (2^{78} for our default key) is not security by itself: the effective strength is set by the best attack, and chaining two linear ciphers lets an attacker solve each stage separately. The suggestions in Section 9 (non-periodic keystream, a non-linear substitution stage and a transposition between stages) address exactly these weaknesses at linear cost, and give a clear path for further work.

References

Shannon, C. E. (1949). Communication Theory of Secrecy Systems. *Bell System Technical Journal*, 28(4), 656–715.

Hill, L. S. (1929). Cryptography in an Algebraic Alphabet. *The American Mathematical Monthly*, 36(6), 306–312.

Friedman, W. F. (1922). *The Index of Coincidence and Its Applications in Cryptography*. Riverbank Laboratories, Publication No. 22.

Stinson, D. R., & Paterson, M. B. (2018). *Cryptography: Theory and Practice* (4th ed.). CRC Press.

Lewand, R. E. (2000). *Cryptological Mathematics*. Mathematical Association of America. (English letter-frequency table.)

Chesterton, G. K. (1908). *The Man Who Was Thursday: A Nightmare*. Project Gutenberg (test plaintext).